

Week 11 - CH08. 텍스트 분석

NLP

- 머신이 인간의 언어를 이해하고 해석하는 데에 중점
 - 언어 해석 위한 기계 번역
 - 질의응답 시스템

Text Mining

- 비정형 텍스트에서 의미 있는 정보를 추출하는 것에 중점
 - 비즈니스 인텔리전스
 - 예측 분석 작업 수행
- 텍스트 분류(Text Classification): Text Categorization이라고도 합니다. 문서가 특정 분류 또는 카테고리에 속하는 것을 예측하는 기법을 통칭합니다. 예를 들어 특정 신문 기사 내용이 연예/정치/사회/문화 중 어떤 카테고리에 속하는지 자동으로 분류하거나 스팸 메일 검출 같은 프로그램이 이에 속합니다. 지도학습을 적용합니다.
- 감성 분석(Sentiment Analysis): 텍스트에서 나타나는 감정/판단/민감/의견/기분 등의 주관적인 요소를 분석하는 기법을 통칭합니다. 소셜 미디어 감정 분석, 영화나 제품에 대한 긍정 또는 리뷰, 여론조사 의견 분석 등의 다양한 영역에서 활용됩니다. Text Analytics에서 가장 활발하게 사용되고 있는 분야입니다. 지도학습 방법뿐만 아니라 비지도학습을 이용해 적용할 수 있습니다.
- 텍스트 요약(Summarization): 텍스트 내에서 중요한 주제나 중심 사상을 추출하는 기법을 말합니다. 대표적으로 토픽 모델링(Topic Modeling)이 있습니다.
- 텍스트 군집화(Clustering)와 유사도 측정: 비슷한 유형의 문서에 대해 군집화를 수행하는 기법을 말합니다. 텍스트 분류를 비지도학습으로 수행하는 방법의 일환으로 사용될 수 있습니다. 유사도 측정 역시 문서들간의 유사도를 측정해 비슷한 문서끼리 모을 수 있는 방법입니다.

01 텍스트 분석 이해

텍스트 분석

- 비정형 데이터인 텍스트 분석

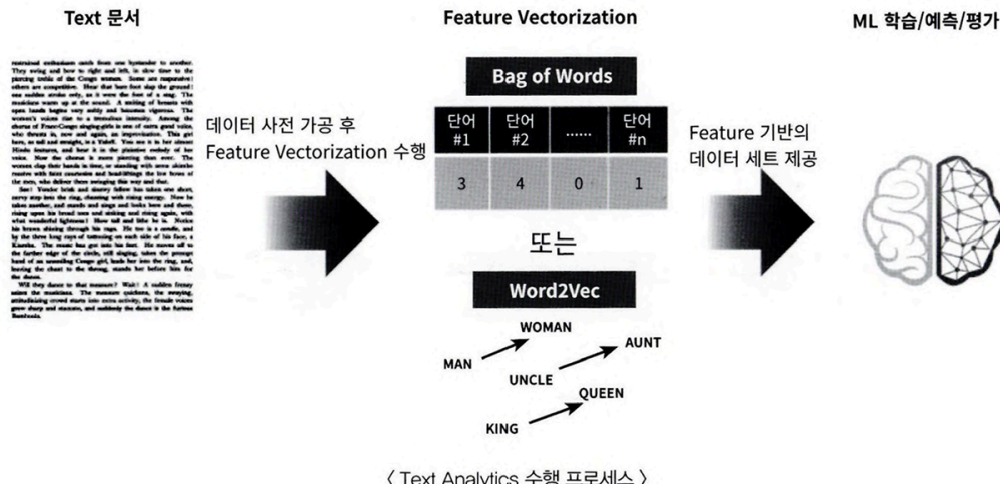
지금까지의 ML 모델

- 주어진 정형 데이터 기반에서 모델 수립, 예측 수행

- ML 알고리즘은 숫자형 feature 기반 데이터만 입력받을 수 있음
 - 텍스트를 머신러닝에 적용하기 위해 feature 추출, 값 부여가 중요
 - Feature Vectorization, Feature Extraction
 - BOW(Bag of Words), Word2Vec

텍스트 분석 수행 프로세스

1. 텍스트 사전 준비 작업 (텍스트 전처리)
 - a. 텍스트를 feature로 만들기 전
 - i. 클렌징 (클렌징, 대/소문자 변경, 특수문자 삭제)
 - ii. 단어 등의 토큰화 작업
 - iii. Stop word(의미 없는 단어) 제거
 - iv. Stemming/Lemmatization(어근 추출)
2. feature vectorization/extraction
 - a. 전처리로 가공된 텍스트에서 feature 추출, 벡터 값 할당
 - i. BOW
 1. Count 기반
 2. TF-IDF 기반
 - ii. Word2Vec
3. ML 모델 수립 및 학습/예측/평가
 - a. feature vectorization된 데이터 세트에 ML 모델 적용
 - i. 학습/예측 및 평가 수행



파이썬 기반의 NLP, 텍스트 분석 패키지

- 쉽게 텍스트 사전 정제 작업, feature vectorization, ML 모델 지원하는 라이브러리 있음
 - NLTK(Natural Language Toolkit for Python)
 - 파이썬 가장 대표적 NLP 패키지
 - 방대한 데이터 세트와 서브 모듈
 - NLP의 거의 모든 영역 커버
 - 많은 NLP가 NLTK 영향 받아 작성됨
 - 수행 속도 느려서 실제 대량 데이터 기반에서 제대로 활용 안 됨
 - Gensim
 - 토픽 모델링 분야에서 가장 두각을 나타냄
 - Word2Vec 구현 등의 신기능 제공
 - SpaCy와 함께 가장 많이 사용되는 NLP 패키지
 - SpaCy
 - 뛰어난 수행 성능
 - 최근 가장 주목 받는 NLP 패키지
 - 사용 사례 늘어나는 중

02 텍스트 사전 준비 작업(텍스트 전처리) - 텍스트 정규화

- 텍스트 자체를 바로 feature로 만들 수 없음
- 사전에 텍스트 가공하는 준비 작업 필요함

텍스트 정규화

- 텍스트를 머신러닝 알고리즘이나 NLP 애플리케이션에 입력 데이터로 사용하기 위해 사전 작업 수행
 - 클렌징(Cleansing)
 - 토큰화(Tokenization)
 - 필터링/스톱 워드 제거/철자 수정
 - Stemming
 - Lemmatization

클렌징

- 텍스트에서 분석에 오히려 방해 되는 요소 제거
 - HTML, XML 태그, 특정 기호 등

텍스트 토큰화

- 문장 토큰화
 - 마침표, 개행문자(\n) 등 문장의 마지막임을 뜻하는 기호에 따라 분리하는 것이 일반적
 - 일반적으로 많이 쓰이는 `sent_tokenize`
 - 3개의 문장으로 이뤄진 텍스트 문서를 문장으로 각각 분리


```

from nltk import sent_tokenize
import nltk
nltk.download('punkt')

import nltk
nltk.download('punkt_tab')

text_sample = 'The Matrix is everywhere its all around us, here even in this room. #
               You can see it out your window or on your television. #
               You feel it when you go to work, or go to church or pay your taxes.'
sentences = sent_tokenize(text=text_sample)
print(type(sentences), len(sentences))
print(sentences)

<class 'list'> 3
['The Matrix is everywhere its all around us, here even in this room.', 'You can see it out your window or on your
[nltk_data] Downloading package punkt to /root/nltk_data...
[nltk_data] Package punkt is already up-to-date!
[nltk_data] Downloading package punkt_tab to /root/nltk_data...
[nltk_data] Unzipping tokenizers/punkt_tab.zip.

```

- sent_tokenize()가 반환하는 것은 각각의 문장으로 구성된 list 객체
 - 반환된 list 객체가 3개의 문장으로 된 문자열을 가지고 있음
- 단어 토큰화
 - 문장을 단어로 토큰화
 - 일반적으로 공백, 콤마, 마침표, 개행 문자 등으로 단어 분리
 - 정규 표현식을 이용해 다양한 유형으로 토큰화 수행 가능
 - 마침표, 개행문자 등 문장 분리 구분자 이용해 단어 토큰화 가능
 - Bag of Word 같이 단어 순서 중요하지 않은 경우 문장 토큰화 사용하지 않고 단어 토큰화만 사용해도 충분
 - 일반적으로 문장 토큰화는 각 문장이 가지는 시맨틱적 의미 중요 요소로 사용될 때 사용
 - word_tokenize()로 단어 토큰화

```

from nltk import word_tokenize

sentence = 'The Matix is everywhere its all around us, here even in this room.'
words = word_tokenize(sentence)
print(type(words), len(words))
print(words)

<class 'list'> 15
['The', 'Matix', 'is', 'everywhere', 'its', 'all', 'around', 'us', ',', 'here', 'even', 'in',

```

- sent_tokenize, word_tokenize 조합하여 문서 모든 단어 토큰화

```

from nltk import word_tokenize, sent_tokenize

def tokenize_text(text):

    sentences = sent_tokenize(text)

    word_tokens = [word_tokenize(sentence) for sentence in sentences]
    return word_tokens

word_tokens = tokenize_text(text_sample)
print(type(word_tokens), len(word_tokens))
print(word_tokens)

<class 'list'> 3
[['The', 'Matrix', 'is', 'everywhere', 'its', 'all', 'around', 'us', ',', 'here', 'even', 'in

```

- 문서를 먼저 문장으로 나누고, 개별 문장을 다시 단어로 토큰화
- 3개의 문장을 문장별로 먼저 토큰화
 - word_tokens 변수는 3개의 리스트 객체 내포하는 리스트
 - 내포된 개별 리스트 객체는 각각 문장별로 토큰화된 단어를 요소로 가짐
- 문장을 단어별로 하나씩 토큰화 하면 문맥적 의미 무시될 수밖에 없음
 - 그래서 도입된 것이 n-gram
 - 연속된 n개의 단어를 하나의 토큰화 단위로 분리해 내는 것
 - n개의 단어 크기 윈도우를 만들어 문장의 처음부터 오른쪽으로 움직이면서 토큰화 수행
 - e.g.) 'Agent Smith knocks the door' 를 2-gram(bigram)으로 만들면
 - (Agent Smith), (Smith knocks), (knocks, the), (the, door)로 토큰화

스톱 워드 제거

- 분석에 큰 의미 없는 단어
- is, the, a, will 등
- 문법 특성으로 빈번히 등장하므로 제거 안 하면 중요 단어로 인지될 수 있음
 - 제거하는 전처리 작업 해야 함

- 언어별로 스톱 워드 목록화되어 있음
- NLTK 경우 가장 다양한 언어의 스톱 워드 제공
- 스톱워드 목록 다운로드

```
import nltk
nltk.download('stopwords')

[nltk_data] Downloading package stopwords to /root/nltk_data...
[nltk_data]   Unzipping corpora/stopwords.zip.
True
```

- 영어의 stopwords 20개 확인

```
print('영어 stop words 개수:', len(nltk.corpus.stopwords.words('english')))
print(nltk.corpus.stopwords.words('english')[:20])

영어 stop words 개수: 198
['a', 'about', 'above', 'after', 'again', 'against', 'ain', 'all', 'am', 'an', 'and', 'any', 'are', 'aren', 'aren'
```

- 영어 스톱워드 198개
- 3개의 문장별로 단어 토큰화해 생성된 word_tokens 리스트에 대해 stopwords를 필터링으로 제거해 분석을 위한 의미 있는 단어만 추출

```
import nltk

stopwords = nltk.corpus.stopwords.words('english')
all_tokens = []

for sentence in word_tokens:
    filtered_words=[]

    for word in sentence:

        word = word.lower()

        if word not in stopwords:
            filtered_words.append(word)
        all_tokens.append(filtered_words)

print(all_tokens)

[['matrix', 'everywhere', 'around', 'us', '.', 'even', 'room', '.'], ['see', 'window', 'televi
```

- is, this 등 스톱 워드가 필터링 통해 제거됨

Stemming과 Lemmatization

- 단어는 문법적 요소에 따라 다양하게 변함

- 영어는 과거/현재, 3인칭 단수 여부, 진행형 등 매우 많은 조건
 - work, works, working, worked
- 문법적 또는 의미적으로 변화하는 단어의 원형을 찾는 것
- Lemmatization
 - 더 정교
 - 의미론적인 기반에서 단어의 원형 찾을
 - 품사 등의 문법적 요소와 더 의미적인 부분을 감안해 정확한 철자로 된 어근 단어 찾을
 - 변환에 더 오랜 시간 걸림
- Stemming
 - 원형 단어로 변환 시 일반적인 방법을 적용 or 더 단순화된 방법 적용
 - 원래 단어에서 일부 철자가 훼손된 어근 단어를 추출하는 경향이 있음
- NLTK는 다양한 Stemmer 제공
 - Porter
 - Lancaster
 - Snowball Stemmer
- NLTK 제공 Lemmatization
 - WordNetLemmatizer
- NLTK의 LancasterStemmer
 - 진행형, 3인칭 단수, 과거형에 따른 동사, 비교, 최상에 따른 형용사 변화에 따라 Stemming은 더 단순하게 원형 단어 찾아줌
 - 필요한 Stemmer객체 생성 뒤 이 객체의 stem('단어') 메서드를 호출하면 원하는 '단어'의 Stemming이 가능

```

from nltk.stem import LancasterStemmer
stemmer = LancasterStemmer()

print(stemmer.stem('working'),stemmer.stem('works'),stemmer.stem('worked'))
print(stemmer.stem('amusing'),stemmer.stem('amuses'),stemmer.stem('amused'))
print(stemmer.stem('happier'),stemmer.stem('happiest'))
print(stemmer.stem('fancier'),stemmer.stem('fanciest'))

work work work
amus amus amus
happy happiest
fant fanciest

```

- work의 경우 working, works 등 모두 단순한 변화이므로 work를 제대로 인식
- amuse의 경우 amus에 ing, s, ed가 붙으므로 amus를 원형으로 인식함
- happy, fancy의 경우 변형된 단어의 정확한 원인 찾지 못하고 원형 단어에서 철자 다른 어근 단어로 인식하는 경우 발생
- WordNetLemmatizer 이용해 Lemmatization 수행
 - 일반적으로 Lemmatization은 보다 정확한 원형 단어 추출을 위해 단어의 '품사' 입력해야 함

```

from nltk.stem import WordNetLemmatizer
import nltk
nltk.download('wordnet')

lemma = WordNetLemmatizer()
print(lemma.lemmatize('amusing','v'),lemma.lemmatize('amuses','v'),lemma.lemmatize('amused',
print(lemma.lemmatize('happier','a'),lemma.lemmatize('happiest','a'))
print(lemma.lemmatize('fancier','a'),lemma.lemmatize('fanciest','a'))

[nltk_data] Downloading package wordnet to /root/nltk_data...
amuse amuse amuse
happy happy
fancy fancy

```

- lemmatize()의 파라미터로 동사의 경우 'v', 형용사는 'a' 입력
- 앞의 Stemmer보다 정확하게 원형 단어 추출해줌

03 Bag of Words -BOW

- 문서가 가지는 모든 단어를 문맥/순서 무시하고 일괄적으로 단어에 대해 빈도 값 부여해 feature 값 추출하는 모델
- 문서 내 모든 단어 한꺼번에 봉투(Bag)에 넣고 섞는다는 의미



〈 Bag of Words 모델 〉

2개의 문장을 Bag of words의 단어 수 기반으로 feature 추출

- My wife likes to watch baseball games and my daughter likes to watch baseball games too
 - My wife likes to play baseball
1. 문장 1과 2에 있는 모든 단어에서 중복 제거
 2. 각 단어 (feature 또는 term)를 칼럼 형태로 나열
 3. 각 단어에 고유의 인덱스 부여
 - a. 'and':0, 'baseball':1, 'daughter':2, 'games':3, 'like':4, 'my':5, 'paly':6, 'to':7, 'too':8, 'watch':9, 'wife':10
 4. 개별 문장에서 해당 단어가 나타나는 횟수(Occurrence)를 각 단어에 기재

	Index 0	Index 1	Index 2	Index 3	Index 4	Index 5	Index 6	Index 7	Index 8	Index 9	Index 10
	and	baseball	daughter	games	likes	my	play	to	too	watch	wife
문장 1	1	2	1	2	2	2		2	1	2	1
문장 2		1			1	1	1	1			1

→ 문장 1에서 baseball은 2회 나타남

BOW 장점

- 쉽고 빠른 구축
- 단어 발생 횟수 기반이지만, 생각보다 문서 특징 잘 나타내므로 여러 분야 활용도 높음

BOW 한계

- 문맥 의미(Semantic Context) 반영 부족
 - 단어 순서 고려하지 않음
 - 문맥적 의미 무시됨
 - 보완 위해 n_gram 기법 사용할 수 있지만 제한적 부분에 그침
 - 언어의 큰 부분인 문맥적 해석 처리 못함
- 희소 행렬 문제(희소성, 희소 행렬)
 - BOW로 feature vectorization 수행하면 희소 행렬 데이터 세트 만들어지기 쉬움
 - 많은 문서 단어 추출 → 매우 많은 단어가 칼럼으로 만들어짐
 - 문서마다 서로 다른 단어로 구성됨
 - 단어가 문서마다 나타나지 않는 경우가 훨씬 더 많음
 - ⇒ 매우 많은 문서에서 단어의 총 개수는 수만~수십만 개가 될 수 있음
 - 하나의 문서에 있는 단어는 극히 일부임
 - 대부분 데이터는 0값으로 채워지게 됨
 - 대규모의 칼럼으로 구성된 행렬에서 대부분 값이 0으로 채워지는 행렬을 희소 행렬(Sparse Matrix)라고 함
 - 반대로 대부분 0이 아닌 의미 있는 값으로 채워지는 행렬을 밀집 행렬(Dense Matrix)라고 함
- 희소 행렬은 일반적으로 ML 수행 시간, 예측 성능 저하시킴
 - 희소 행렬 위한 특별한 기법 존재

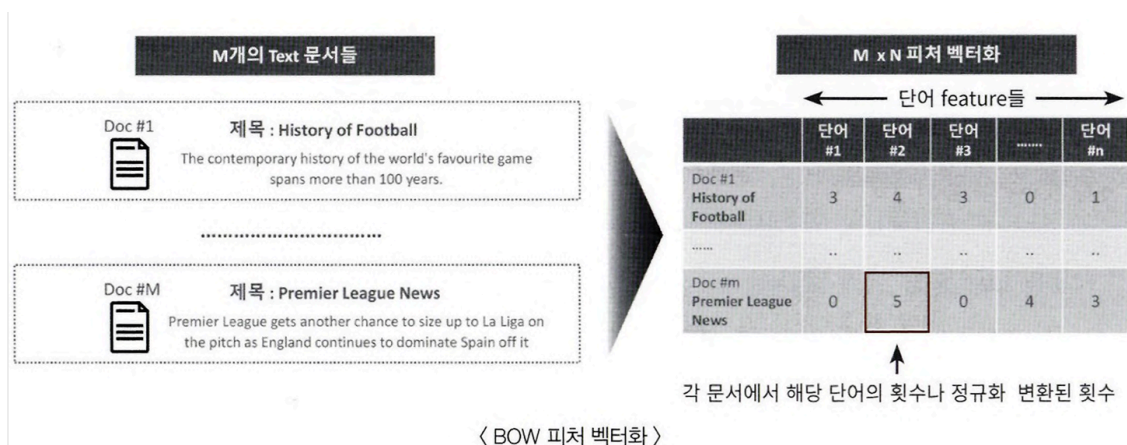
BOW feature vectorization

ML 알고리즘

- 일반적으로 숫자형 feature를 데이터로 입력 받아 동작
 - 텍스트 같은 데이터는 바로 입력할 수 없음
 - 텍스트는 특정 의미 가지는 숫자형 값인 벡터 값으로 변환해야 함
 - = feature vectorization
 - 각 문서의 텍스트를 단어로 추출해 피처로 할당
 - 각 단어의 발생 빈도와 같은 값을 이 피처에 값으로 부여해 각 문서를 이 단어 피처 발생 빈도 값으로 구성된 벡터로 만드는 기법
 - 기존 텍스트 데이터를 또 다른 형태의 피처의 조합으로 변경
 - 넓은 범위의 피처 추출에 포함함
 - Text Analysis에서는 피처 벡터와 피처 추출을 같은 의미로 사용

BOW 모델의 피처 벡터화

- 모든 문서에서 모든 단어를 칼럼 형태로 나열
- 각 문서에서 해당 단어의 횟수나 정규화된 빈도를 값으로 부여하는 데이터 세트 모델로 변경하는 것
- M개의 텍스트 문서가 있음
 - 문서에서 모든 단어 추출 나열 → N개의 단어
 - 문서의 피처 벡터화 수행하면 M x N개의 단어 피처로 이뤄진 행렬 구성



BOW의 feature vectorization

- 카운트 기반의 벡터화
- TF-IDF(Term Frequency - Inverse Document Frequency)

카운트 벡터화

- 단어 피처에 값 부여할 때 각 문서에서 해당 단어가 나타나는 횟수, 즉 Count를 부여하는 경우
- 카운트 값 높을수록 중요 단어
- 카운트만 부여하면 문서의 특징을 나타낸다고보다 언어의 특성상 문장에서 자주 사용될 수밖에 없는 단어까지 높은 값을 부여하게 됨
 - 이 문제 보완 위해 TF-IDF 벡터화 사용

TF-IDF

- 개별 문서에서 자주 나타나는 단어에 높은 가중치
- 모든 문서에서 전반적으로 자주 나타나는 단어에 대해서는 패널티 줌
- 어떤 문서에서 특정 단어 자주 나타나면 그 단어는 해당 문서를 특징 짓는 중요 단어일 수 있음
- 다른 문서에서도 자주 나타나는 단어라면 언어 특성상 범용적으로 자주 사용되는 단어일 가능성 높음
- 모든 문서에서 반복적으로 자주 발생하는 단어에는 패널티 부여해서 가중치 균형 맞춤
- 텍스트 길고 문서 개수 많다면 카운트보다 TF-IDF방식이 예측 성능 보장해줌



한 개의 문서(Document)



모든 문서들(Corpus)

The	Matrix	is	nothing	but	an	advertising	gimmick
40	5	50	12	20	45	3	2

The	Matrix	is	nothing	but	an	advertising	gimmick
2000	190	2300	500	1200	3000	52	12

TF_i = 개별 문서에서의 단어 i 빈도

DF_i = 단어 i를 가지고 있는 문서 개수

N = 전체 문서 개수

$$TFIDF_i = TF_i * \log \frac{N}{DF_i}$$

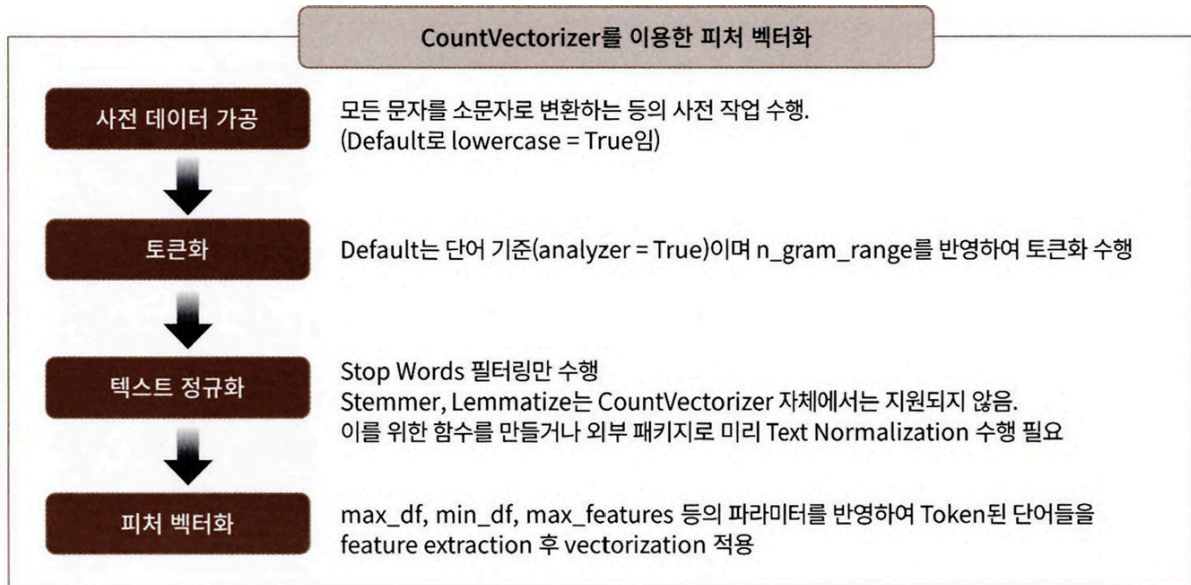
사이킷런의 Count 및 TF-IDF 벡터화 구현: CountVectorizer, TfidfVectorizer

- 카운트 기반의 벡터화 구현한 클래스
- 단지 피처 벡터화만 수행하지는 않으며 소문자 일괄 변환, 토큰화, 스톱 워드 필터링 등의 텍스트 전처리도 함께 수행
- 입력 파라미터를 설정해 동작
- fit(), transform() 통해 피처 벡터화된 객체 반환
- 파라미터

파라미터 명	파라미터 설명
max_df	전체 문서에 걸쳐서 너무 높은 빈도수를 가지는 단어 피처를 제외하기 위한 파라미터입니다. 너무 높은 빈도수를 가지는 단어는 스톱 워드와 비슷한 문법적인 특성으로 반복적인 단어일 가능성이 높기에 이를 제거하기 위해 사용됩니다. max_df = 100과 같이 정수 값을 가지면 전체 문서에 걸쳐 100개 이하로 나타나는 단어만 피처로 추출합니다. Max_df = 0.95와 같이 부동소수점 값(0.0 ~ 1.0)을 가지면 전체 문서에 걸쳐 빈도수 0~95%까지의 단어만 피처로 추출하고 나머지 상위 5%는 피처로 추출하지 않습니다.
min_df	전체 문서에 걸쳐서 너무 낮은 빈도수를 가지는 단어 피처를 제외하기 위한 파라미터입니다. 수백~수천 개의 전체 문서에서 특정 단어가 min_df에 설정된 값보다 적은 빈도수를 가진다면 이 단어는 크게 중요하지 않거나 가비지(garbage)성 단어일 확률이 높습니다. min_df = 2와 같이 정수 값을 가지면 전체 문서에 걸쳐서 2번 이하로 나타나는 단어는 피처로 추출하지 않습니다. min_df = 0.02와 같이 부동소수점 값(0.0 ~ 1.0)을 가지면 전체 문서에 걸쳐서 하위 2% 이하의 빈도수를 가지는 단어는 피처로 추출하지 않습니다.
max_features	추출하는 피처의 개수를 제한하며 정수로 값을 지정합니다. 가령 max_features = 2000으로 지정할 경우 가장 높은 빈도를 가지는 단어 순으로 정렬해 2000개까지만 피처로 추출합니다.
stop_words	'english'로 지정하면 영어의 스톱 워드로 지정된 단어는 추출에서 제외합니다.
ngram_range	Bag of Words 모델의 단어 순서를 어느 정도 보강하기 위한 n_gram 범위를 설정합니다. 튜플 형태로 (범위 최솟값, 범위 최댓값)을 지정합니다. 예를 들어 (1, 1)로 지정하면 토큰화된 단어를 1개씩 피처로 추출합니다. (1, 2)로 지정하면 토큰화된 단어를 1개씩(minimum 1), 그리고 순서대로 2개씩(maximum 2) 묶어서 피처로 추출합니다.
analyzer	피처 추출을 수행한 단위를 지정합니다. 당연히 디폴트는 'word'입니다. Word가 아니라 character의 특정 범위를 피처로 만드는 특정한 경우 등을 적용할 때 사용됩니다.
token_pattern	토큰화를 수행하는 정규 표현식 패턴을 지정합니다. 디폴트 값은 '\b\w+\b'로, 공백 또는 개행 문자 등으로 구분된 단어 분리자(\b) 사이의 2문자(문자 또는 숫자, 즉 영숫자) 이상의 단어(word)를 토큰으로 분리합니다. analyzer= 'word'로 설정했을 때만 변경 가능하나 디폴트 값을 변경할 경우는 거의 발생하지 않습니다.
tokenizer	토큰화를 별도의 커스텀 함수로 이용시 적용합니다. 일반적으로 CountTokenizer 클래스에서 어근 변환 시 이를 수행하는 별도의 함수를 tokenizer 파라미터에 적용하면 됩니다.

- 영어의 경우 모든 문자를 소문자로 변경
- 디폴트로 단어 기준으로 n_gram_range 반영해 각 단어를 토큰화
- 텍스트 정규화 수행

- stop_words = 'english'같이 stop_words 파라미터가 주어진 경우 스톱 워드 필터링만 가능
- Stemming과 Lemmatization 같은 어근 변환은 직접 지원하지 않으나 tokenizer 파라미터에 커스텀 어근 변환 함수를 적용하여 어근 변환 수행 가능
- max_df, min_df, max_features 등의 파라미터 이용해 토큰화된 단어를 피처로 추출, 단어 빈도수 벡터 값을 적용



BOW 벡터화를 위한 희소 행렬

- 사이킷런의 CountVectorizer/TfidfVectorizer 이용해 텍스트를 feature 단위로 벡터화해 변환
- CSR 형태의 희소 행렬을 반환
- 대규모 행렬의 대부분의 값을 0이 차지하는 행렬을 희소 행렬이라 함
- BOW 형태를 가진 언어 모델의 피처 벡터화는 대부분 희소 행렬

←
→
수십만 개의 칼럼

	단어 1	단어 2	단어 3		단어 1000		단어 2000		단어 10000		단어 20000		단어 100000
수천~ 수만 개 레코드	문서1	1	2	2	0	0	0	0	0	0	0	0	0
	문서2	0	0	1	0	0	1	0	0	1	0	0	1
	문서 ...												
	문서 10000	0	1	3	0	0	0	0	0	0	0	0	0

BOW의 Vectorization 모델은 너무 많은 0값이 메모리 공간에 할당되어 많은 메모리 공간이 필요하며
연산 시에도 데이터 액세스를 위한 많은 시간이 소모됩니다.

- 희소 행렬은 너무 많은 불필요한 0값이 메모리 공간에 할당 되어 메모리 공간 많이 필요 함
- 행렬 크기 커서 연산 시에도 데이터 액세스 위한 시간 많이 소모됨
- 물리적으로 적은 메모리 공간 차지할 수 있게 변환
 - COO 형식
 - CRS 형식

희소 행렬 - COO 형식

- 0이 아닌 데이터만 별도의 데이터 배열에 저장
- 그 데이터가 가리키는 행과 열의 위치를 별도의 배열로 저장하는 방식
- $[[3, 0, 1], [0, 2, 0]]$ 같은 2차원 데이터 있다 하면
 - 0이 아닌 데이터는 $[3, 1, 2]$
 - 0이 아닌 데이터가 있는 위치를 (row, col)로 표현한다면
 - $(0, 0), (0, 2), (1, 1)$
 - 로우와 칼럼을 별도의 배열로 저장하면 로우는 $[0, 0, 1]$ 이고 칼럼은 $[0, 2, 1]$
- 파이썬은 희소 행렬 변화 위해 사이파이 이용
- 사이파이의 sparse 패키지는 희소 행렬 반환 위한 모듈 제공

```
import numpy as np

dense = np.array( [ [ 3, 0, 1 ], [0, 2, 0 ] ] )
```

```
from scipy import sparse

data = np.array([3,1,2])

row_pos = np.array([0,0,1])
col_pos = np.array([0,2,1])

sparse_coo = sparse.coo_matrix((data, (row_pos,col_pos)))
```

- sparse_coo는 COO 형식의 희소 행렬 객체 변수

toarray() 메서드 이용해 다시 밀집 형태의 행렬로 출력

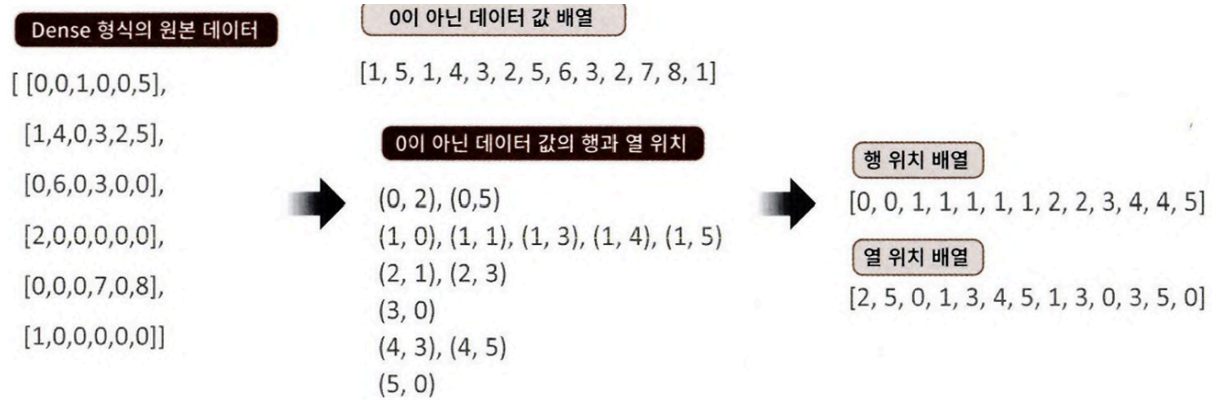
```
sparse_coo.toarray()

array([[3, 0, 1],
       [0, 2, 0]])
```

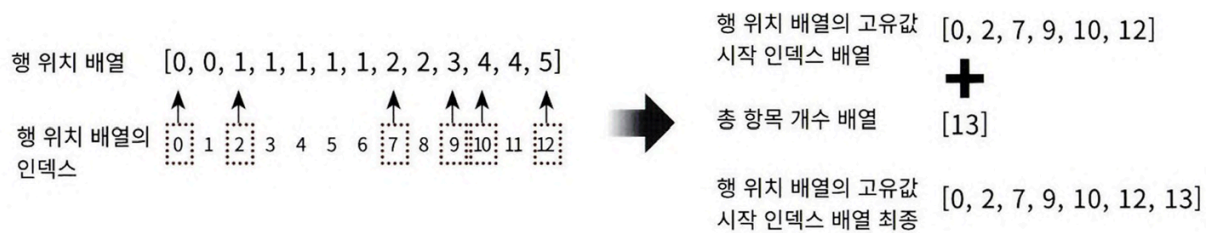
- 다시 원래의 데이터 행렬로 추출됨

희소 행렬 - CRS 형식

- COO 형식이 행과 열의 위치를 나타내기 위해 반복적인 위치 데이터를 사용해야 하는 문제점 해결 방식
- COO 변환 형식의 문제점



- 행 위치 배열이 0부터 순차적 증가하는 값으로 이뤄졌다는 특성 고려하면 행 위치 배열의 고유 값의 시작 위치만 표기하는 방법으로 반복 제거 가능
- 행 위치 배열 내에 있는 고유 값의 시작 위치만 다시 별도의 위치 배열로 가지는 변환 방식
- 고유 값의 시작 위치만 알고 있으면 얼마든 행 위치 배열을 다시 만들 수 있게 COO 방식보다 메모리 적게 들고 빠른 연산 가능



- `crs_matrix` 클래스 이용해 가능


```

from scipy import sparse

dense2 = np.array([[0,0,1,0,0,5],
                   [1,4,0,3,2,5],
                   [0,6,0,3,0,0],
                   [2,0,0,0,0,0],
                   [0,0,0,7,0,8],
                   [1,0,0,0,0,0]])

# 0 이 아닌 데이터 추출
data2 = np.array([1, 5, 1, 4, 3, 2, 5, 6, 3, 2, 7, 8, 1])

# 행 위치와 열 위치를 각각 array로 생성
row_pos = np.array([0, 0, 1, 1, 1, 1, 1, 2, 2, 3, 4, 4, 5])
col_pos = np.array([2, 5, 0, 1, 3, 4, 5, 1, 3, 0, 3, 5, 0])

# COO 형식으로 변환
sparse_coo = sparse.coo_matrix((data2, (row_pos,col_pos)))

# 행 위치 배열의 고유한 값들의 시작 위치 인덱스를 배열로 생성
row_pos_ind = np.array([0, 2, 7, 9, 10, 12, 13])

# CSR 형식으로 변환
sparse_csr = sparse.csr_matrix((data2, col_pos, row_pos_ind))

print('COO 변환된 데이터가 제대로 되었는지 다시 Dense로 출력 확인')
print(sparse_coo.toarray())
print('CSR 변환된 데이터가 제대로 되었는지 다시 Dense로 출력 확인')
print(sparse_csr.toarray())

```

```

COO 변환된 데이터가 제대로 되었는지 다시 Dense로 출력 확인
[[0 0 1 0 0 5]
 [1 4 0 3 2 5]
 [0 6 0 3 0 0]
 [2 0 0 0 0 0]
 [0 0 0 7 0 8]
 [1 0 0 0 0 0]]
CSR 변환된 데이터가 제대로 되었는지 다시 Dense로 출력 확인
[[0 0 1 0 0 5]
 [1 4 0 3 2 5]
 [0 6 0 3 0 0]
 [2 0 0 0 0 0]
 [0 0 0 7 0 8]
 [1 0 0 0 0 0]]

```

05 감정 분석

- 문서의 주관적 감성/의견/감정/기분 등 파악 위한 방법
- 문서 내 텍스트가 나타내는 여러 주관적 단어/문맥 기반으로 감성 수치 계산 방법 이용
- 감정 지수
 - 긍정 감정 지수
 - 부정 감정 지수
 - 두 지수 합산해 긍정 or 부정 감성 결정
- 지도 학습
 - 학습 데이터와 타겟 레이블 값을 기반으로 감성 분석 학습 수행
 - 이를 기반으로 다른 데이터의 감성 분석 예측하는 방법으로 일반적 텍스트 기반 분류와 거의 동일
- 비지도 학습
 - Lexicon 이라는 일종의 감성 어휘 사전 이용
 - Lexicon은 감성 분석 위한 용어와 문맥에 대한 다양한 정보 가짐
 - 이를 이용해 문서의 긍/부정 감성 여부 판단

지도학습 기반 감성 분석 실습 - IMDB 영화평

```
import pandas as pd

review_df = pd.read_csv('./labeledTrainData.tsv', header=0, sep="\t", quoting=3)
review_df.head(3)
```

	id	sentiment	review
0	"5814_8"	1	"With all this stuff going down at the moment ...
1	"2381_9"	1	"\"The Classic War of the Worlds\" by Timothy ...
2	"7759_3"	0	"The film starts with a manager (Nicholas Bell...

```
print(review_df['review'][0])

"With all this stuff going down at the moment with MJ i've started listening to his music, watching the odd docum
```


- HTML 형식에서 추출해
 태그가 존재
 - feature로 만들 필요 없으니 삭제
 - DataFrame/Series는 문자열 연산 지원 위해 str thrtjd dldyd
 - replace() 적용
- 영어 아닌 숫자/특수 문자 역시 Sentiment 위한 feature로는 의미 없어 보임
 - 공란으로 변경
- 숫자/특수문자 찾고 변환
 - 정규 표현식 이용
- 파이썬의 re 모듈
 - 정규 표현식 지원
- 정규 표현식 [^a-zA-Z]
 - 영어 대/소문자 아닌 모든 문자 찾는 것
- re.sub("[^a-zA-Z]", "", z)
 - 영어 대/소문자 아닌 모든 문자 찾아서 공란으로 변경
- DataFrame에 re.sub()는 lambda 식을 이용해 적용

```
import re

# <br> html 태그는 replace 함수로 공백으로 변환
review_df['review'] = review_df['review'].str.replace('<br />', ' ')

# 파이썬의 정규 표현식 모듈인 re를 이용하여 영어 문자열이 아닌 문자는 모두 공백으로 변환
review_df['review'] = review_df['review'].apply( lambda x : re.sub("[^a-zA-Z]", " ", x) )
```

- 결정값 클래스인 sentiment 칼럼을 별도로 추출
 - 결정 값 데이터 세트 생성
 - 원본 데이터 세트에서 id와 sentiment 칼럼 삭제해 피쳐 데이터 세트 생성
 - train_test_split() 이용해 학습용, 테스트용 데이터 세트 분리

```

from sklearn.model_selection import train_test_split

class_df = review_df['sentiment']
feature_df = review_df.drop(['id', 'sentiment'], axis=1, inplace=False)

X_train, X_test, y_train, y_test = train_test_split(feature_df, class_df, test_size=0.3, random_state=156)

X_train.shape, X_test.shape

((17500, 1), (7500, 1))

```

- 감상평 텍스트 피쳐 벡터화한 후 ML 분류 알고리즘 적용
 - 예측 성능 측정
- Pipeline 객체 이용해 두 가지 한꺼번에 수행
- Classifier는 LogisticRegression을 이용
- Count 벡터화

```

from sklearn.feature_extraction.text import CountVectorizer, TfidfVectorizer
from sklearn.pipeline import Pipeline
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score, roc_auc_score

# 스톱 워드는 English, filtering, ngram은 (1,2)로 설정해 CountVectorization수행.
# LogisticRegression의 C는 10으로 설정.
pipeline = Pipeline([
    ('cnt_vect', CountVectorizer(stop_words='english', ngram_range=(1,2) )),
    ('lr_clf', LogisticRegression(solver='liblinear', C=10))]

# Pipeline 객체를 이용하여 fit(), predict()로 학습/예측 수행. predict_proba()는 roc_auc때문에 수행.
pipeline.fit(X_train['review'], y_train)
pred = pipeline.predict(X_test['review'])
pred_probs = pipeline.predict_proba(X_test['review'])[:,1]

print('예측 정확도는 {0:.4f}, ROC-AUC는 {1:.4f}'.format(accuracy_score(y_test, pred),
                                                              roc_auc_score(y_test, pred_probs)))

예측 정확도는 0.8861, ROC-AUC는 0.9503

```

- TF-IDF 벡터화
- Pipeline에서 CountVectorizer를 TfidfVectorizer로 변경하면 됨

```
# 스톱 워드는 english, filtering, ngram은 (1,2)로 설정해 TF-IDF 벡터화 수행.
# LogisticRegression의 C는 10으로 설정.
pipeline = Pipeline([
    ('tfidf_vect', TfidfVectorizer(stop_words='english', ngram_range=(1,2) )),
    ('lr_clf', LogisticRegression(solver='liblinear', C=10)))

pipeline.fit(X_train['review'], y_train)
pred = pipeline.predict(X_test['review'])
pred_probs = pipeline.predict_proba(X_test['review'])[:,1]

print('예측 정확도는 {0:.4f}, ROC-AUC는 {1:.4f}'.format(accuracy_score(y_test, pred),
                                                             roc_auc_score(y_test, pred_probs)))

예측 정확도는 0.8936, ROC-AUC는 0.9598
```

- TF-IDF 기반 feature 벡터화의 예측 성능 나아짐

비지도학습 기반 감성 분석 소개

- Lexicon 기반
 - 감정 어휘 사전
 - 감정 지수
 - 단어의 위치나 주변 단어, 문맥, POS 등을 참고해 결정
 - NLTK 패키지가 구현
 - WordNet
 - 방대한 영어 어휘 사전
 - 시맨틱 제공하는 어휘 사전
 - 문맥상 의미
 - 시맨틱을 프로그램적으로 인터페이스할 수 있는 방법 제공
 - 품사로 구성된 개별 단어를 Synset(Sets of cognitive synonyms)라는 개념 이용해 표현
 - 하나의 단어가 아니라 그 단어가 가지는 문맥, 시맨틱 정보 제공하는 WordNet의 핵심 개념
- NLTK 감성사전 성능 낮음
 - 실제 업무 적용은 다른 감성 사전 적용함
 - SentiWordNet

- NLTK WordNet와 유사하게 감성 단어 전용의 WordNet 구현
- WordNet의 Synset 개념을 감성 분석에 적용한 것
- Synset별로 3가지 감성 점수 할당
 - 긍정 감성 지수
 - 부정 감성 지수
 - 객관성 지수
- VADER
 - 주로 소셜 미디어 텍스트에 대한 감성 분석 제공
 - 뛰어난 감성 분석 결과, 비교적 빠른 수행 시간
 - 대용량 텍스트에 잘 사용
- Pattern
 - 예측 성능에서 가장 주목 받음

SentiWordNet을 이용한 감성 분석

- NLTK 데이터 세트 다운

```

import nltk
nltk.download('all')

[nltk_data] Downloading collection 'all'
[nltk_data] |
[nltk_data] | Downloading package abc to /root/nltk_data...
[nltk_data] | Unzipping corpora/abc.zip.
[nltk_data] | Downloading package alpino to /root/nltk_data...
[nltk_data] | Unzipping corpora/alpino.zip.
[nltk_data] | Downloading package averaged_perceptron_tagger to
[nltk_data] | /root/nltk_data...
[nltk_data] | Unzipping taggers/averaged_perceptron_tagger.zip.
[nltk_data] | Downloading package averaged_perceptron_tagger_eng to
[nltk_data] | /root/nltk_data...
[nltk_data] | Unzipping
[nltk_data] | taggers/averaged_perceptron_tagger_eng.zip.
[nltk_data] | Downloading package averaged_perceptron_tagger_ru to
[nltk_data] | /root/nltk_data...
[nltk_data] | Unzipping
[nltk_data] | taggers/averaged_perceptron_tagger_ru.zip.
[nltk_data] | Downloading package averaged_perceptron_tagger_rus to
[nltk_data] | /root/nltk_data...
[nltk_data] | Unzipping
[nltk_data] | taggers/averaged_perceptron_tagger_rus.zip.
[nltk_data] | Downloading package basque_grammars to
[nltk_data] | /root/nltk_data...
[nltk_data] | Unzipping grammars/basque_grammars.zip.
[nltk_data] | Downloading package bcp47 to /root/nltk_data...
[nltk_data] | Downloading package biocreative_ppi to
[nltk_data] | /root/nltk_data...
[nltk_data] | Unzipping corpora/biocreative_ppi.zip.
[nltk_data] | Downloading package blip_wsj_no_aux to
[nltk_data] | /root/nltk_data...
[nltk_data] | Unzipping models/blip_wsj_no_aux.zip.
[nltk_data] | Downloading package book_grammars to
[nltk_data] | /root/nltk_data...
[nltk_data] | Unzipping grammars/book_grammars.zip.
[nltk_data] | Downloading package brown to /root/nltk_data...
[nltk_data] | Unzipping corpora/brown.zip.
[nltk_data] | Downloading package brown_tei to /root/nltk_data...
[nltk_data] | Unzipping corpora/brown_tei.zip.
[nltk_data] | Downloading package cess_cat to /root/nltk_data...
[nltk_data] | Unzipping corpora/cess_cat.zip.
[nltk_data] | Downloading package cess_esp to /root/nltk_data...
[nltk_data] | Unzipping corpora/cess_esp.zip.
[nltk_data] | Downloading package chat80 to /root/nltk_data...
[nltk_data] | Unzipping corpora/chat80.zip.
[nltk_data] | Downloading package city_database to
[nltk_data] | /root/nltk_data...

```

- WordNet 모듈 임포트하여 'present' 단어에 대한 Synset 추출
- WordNet의 synsets()는 파라미터로 지정된 단어에 대해 등재된 모든 Synset 객체 반환

```
from nltk.corpus import wordnet as wn

term = 'present'

# 'present'라는 단어로 wordnet의 synsets 생성.
synsets = wn.synsets(term)
print('synsets() 반환 type :', type(synsets))
print('synsets() 반환 값 갯수:', len(synsets))
print('synsets() 반환 값 :', synsets)

synsets() 반환 type : <class 'list'>
synsets() 반환 값 갯수: 18
synsets() 반환 값 : [Synset('present.n.01'), Synset('present.n.02'), Synset('present.n.03'), Synset('show.v.01')]
```

- 호출 시 반환되는 건 Synset 객체를 가지는 리스트
- 총 18개의 서로 다른 semantic을 가지는 synset 객체 반환됨
- present.n.01은 POS 태그 나타냄
 - n: noun, 01은 명사로서 의미 구분 인덱스
- Synset 객체가 가지는 속성 확인
- 정의, 부명제 등으로 시맨틱적 요소 표현 가능

```

for synset in synsets :
    print('#### Synset name : ', synset.name(),'####')
    print('POS : ',synset.lexname())
    print('Definition:',synset.definition())
    print('Lemmas:',synset.lemma_names())

#### Synset name :  present.n.01 ####
POS : noun.time
Definition: the period of time that is happening now; any continuous stretch of time including the moment of speaking
Lemmas: ['present', 'nowadays']
#### Synset name :  present.n.02 ####
POS : noun.possession
Definition: something presented as a gift
Lemmas: ['present']
#### Synset name :  present.n.03 ####
POS : noun.communication
Definition: a verb tense that expresses actions or states at the time of speaking
Lemmas: ['present', 'present_tense']
#### Synset name :  show.v.01 ####
POS : verb.perception
Definition: give an exhibition of to an interested audience
Lemmas: ['show', 'demo', 'exhibit', 'present', 'demonstrate']
#### Synset name :  present.v.02 ####
POS : verb.communication
Definition: bring forward and present to the mind
Lemmas: ['present', 'represent', 'lay_out']
#### Synset name :  stage.v.01 ####
POS : verb.creation
Definition: perform (a play), especially on a stage
Lemmas: ['stage', 'present', 'represent']
#### Synset name :  present.v.04 ####
POS : verb.possession
Definition: hand over formally
Lemmas: ['present', 'submit']
#### Synset name :  present.v.05 ####
POS : verb.stative
Definition: introduce
Lemmas: ['present', 'pose']
#### Synset name :  award.v.01 ####
POS : verb.possession
Definition: give, especially as an honor or reward
Lemmas: ['award', 'present']
#### Synset name :  give.v.08 ####
POS : verb.possession
Definition: give as a present; make a gift of
Lemmas: ['give', 'gift', 'present']
#### Synset name :  deliver.v.01 ####

```

- 어떤 어휘와 다른 어휘 간의 관계를 유사도로 나타낼 수 있음
- path_similarity() 메서드 제공

```
# synset 객체를 단어별로 생성합니다.
tree = wn.synset('tree.n.01')
lion = wn.synset('lion.n.01')
tiger = wn.synset('tiger.n.02')
cat = wn.synset('cat.n.01')
dog = wn.synset('dog.n.01')

entities = [tree, lion, tiger, cat, dog]
similarities = []
entity_names = [entity.name().split('.')[0] for entity in entities]

# 단어별 synset 들을 iteration 하면서 다른 단어들의 synset과 유사도를 측정합니다.
for entity in entities:
    similarity = [round(entity.path_similarity(compared_entity), 2) for compared_entity in entities]
    similarities.append(similarity)

# 개별 단어별 synset과 다른 단어의 synset과의 유사도를 DataFrame형태로 저장합니다.
similarity_df = pd.DataFrame(similarities, columns=entity_names, index=entity_names)
similarity_df
```

	tree	lion	tiger	cat	dog
tree	1.00	0.07	0.07	0.08	0.12
lion	0.07	1.00	0.33	0.25	0.17
tiger	0.07	0.33	1.00	0.25	0.17
cat	0.08	0.25	0.25	1.00	0.20
dog	0.12	0.17	0.17	0.20	1.00

	tree	lion	tiger	cat	dog
tree	1.00	0.07	0.07	0.08	0.12
lion	0.07	1.00	0.33	0.25	0.17
tiger	0.07	0.33	1.00	0.25	0.17
cat	0.08	0.25	0.25	1.00	0.20
dog	0.12	0.17	0.17	0.20	1.00

← lion은 tree와의 유사도가 0.07로 가장 적고, tiger와는 유사도가 0.33으로 가장 큼.

- senti_synsets()는 WordNet 모듈이라 synsets()와 비슷하게 Senti_Synset 클래스를 리스트 형태로 반환

```
import nltk
from nltk.corpus import sentiwordnet as swn

senti_synsets = list(swn.senti_synsets('slow'))
print('senti_synsets() 반환 type :', type(senti_synsets))
print('senti_synsets() 반환 값 갯수:', len(senti_synsets))
print('senti_synsets() 반환 값 :', senti_synsets)

senti_synsets() 반환 type : <class 'list'>
senti_synsets() 반환 값 갯수: 11
senti_synsets() 반환 값 : [SentiSynset('decelerate.v.01'), SentiSynset('slow.v.02'), SentiSynset('slow.v.03'), Ser
```


- 객관성 지수 = 1, 감성 지수 = 0

```
import nltk
from nltk.corpus import sentiwordnet as swn

father = swn.senti_synset('father.n.01')
print('father 긍정감성 지수: ', father.pos_score())
print('father 부정감성 지수: ', father.neg_score())
print('father 객관성 지수: ', father.obj_score())
print('ㄷㄷㄷ')
fabulous = swn.senti_synset('fabulous.a.01')
print('fabulous 긍정감성 지수: ',fabulous .pos_score())
print('fabulous 부정감성 지수: ',fabulous .neg_score())
```

father 긍정감성 지수: 0.0
father 부정감성 지수: 0.0
father 객관성 지수: 1.0

fabulous 긍정감성 지수: 0.875
fabulous 부정감성 지수: 0.125

SentiWordNet 이용한 영화 감상평 감성 분석

1. 문서를 문장 단위로 분해
 2. 문장을 단위로 토큰화하고 품사 태깅
 3. 품사 태깅된 단어 기반으로 synset 객체와 senti_synset 객체를 생성
 4. senti_synset 객체에서 긍/부정 감성 지수 구하고 모두 합산해 특정 임계치 값 이상일 때 긍정 감성, 그렇지 않으면 부정 감성
- WordNet 이용해 단어로 토큰화한 뒤 어근 추출과 품사 태깅 적용

```
from nltk.corpus import wordnet as wn

# 간단한 NLTK PennTreebank Tag를 기반으로 WordNet기반의 품사 Tag로 변환
def penn_to_wn(tag):
    if tag.startswith('J'):
        return wn.ADJ
    elif tag.startswith('N'):
        return wn.NOUN
    elif tag.startswith('R'):
        return wn.ADV
    elif tag.startswith('V'):
        return wn.VERB
    return
```

- 문장→단어 토큰→품사 태깅
- SentiSynset 클래스 생성하고 Polarity Score 합산 함수 생성

```

from nltk.stem import WordNetLemmatizer
from nltk.corpus import sentiwordnet as swn
from nltk import sent_tokenize, word_tokenize, pos_tag

def swn_polarity(text):
    # 감성 지수 초기화
    sentiment = 0.0
    tokens_count = 0

    lemmatizer = WordNetLemmatizer()
    raw_sentences = sent_tokenize(text)
    # 분해된 문장별로 단어 토큰 -> 품사 태깅 후에 SentiSynset 생성 -> 감성 지수 합산
    for raw_sentence in raw_sentences:
        # NLTK 기반의 품사 태깅 문장 추출
        tagged_sentence = pos_tag(word_tokenize(raw_sentence))
        for word, tag in tagged_sentence:

            # WordNet 기반 품사 태깅과 어근 추출
            wn_tag = penn_to_wn(tag)
            if wn_tag not in (wn.NOUN, wn.ADJ, wn.ADV):
                continue
            lemma = lemmatizer.lemmatize(word, pos=wn_tag)
            if not lemma:
                continue
            # 어근을 추출한 단어와 WordNet 기반 품사 태깅을 입력해 Synset 객체를 생성.
            synsets = wn.synsets(lemma, pos=wn_tag)
            if not synsets:
                continue
            # sentiwordnet의 감성 단어 분석으로 감성 synset 추출
            # 모든 단어에 대해 긍정 감성 지수는 +로 부정 감성 지수는 -로 합산해 감성 지수 계산.
            synset = synsets[0]
            swn_synset = swn.senti_synset(synset.name())
            sentiment += (swn_synset.pos_score() - swn_synset.neg_score())
            tokens_count += 1

    if not tokens_count:
        return 0

    # 총 score가 0 이상일 경우 긍정(Positive) 1, 그렇지 않을 경우 부정(Negative) 0 반환
    if sentiment >= 0:
        return 1

    return 0

```

- swn_polarity(text) 함수를 IMDB 감상평의 개별 문서에 적용해 긍/부정 감성 예측
- 판다스의 apply lambda 구문 이용해 적용

```

review_df['preds'] = review_df['review'].apply( lambda x : swn_polarity(x) )
y_target = review_df['sentiment'].values
preds = review_df['preds'].values

```

- SentiWordNet의 감성 분석 예측 성능

```

from sklearn.metrics import accuracy_score, confusion_matrix, precision_score
from sklearn.metrics import recall_score, f1_score, roc_auc_score
import numpy as np

print(confusion_matrix(y_target, preds))
print("정확도:", np.round(accuracy_score(y_target, preds), 4))
print("정밀도:", np.round(precision_score(y_target, preds), 4))
print("재현율:", np.round(recall_score(y_target, preds), 4))

[[7669 4831]
 [3635 8865]]
정확도: 0.6614
정밀도: 0.6473
재현율: 0.7092

```

- 만족스러운 수치는 아님

```

from nltk.sentiment.vader import SentimentIntensityAnalyzer

senti_analyzer = SentimentIntensityAnalyzer()
senti_scores = senti_analyzer.polarity_scores(review_df['review'][0])
print(senti_scores)

{'neg': 0.13, 'neu': 0.743, 'pos': 0.127, 'compound': -0.7943}

```

VADER를 이용한 감성 분석

- SentimentIntensityAnalyzer 객체를 생성한 뒤 문서별로 polarity_scores() 메서드 호출해 감성 점수 구함
- 해당 문서의 감성 점수가 특정 임계값 이상이면 긍정, 아니면 부정
- polarity_scores() 메서드는 딕셔너리 형태의 감성 점수 반환
 - neg: 부정, pos: 긍정, neu: 중립
 - compound: neg, neu, pos 조합하여 -1과 1 사이의 감성지수 표현한 값
 - 보통 0.1이상이면 긍정, 그 이하면 부정

- 텍스트별로 vader_polarity()함수 호출

```
def vader_polarity(review, threshold=0.1):
    analyzer = SentimentIntensityAnalyzer()
    scores = analyzer.polarity_scores(review)

    # compound 값에 기반하여 threshold 입력값보다 크면 1, 그렇지 않으면 0을 반환
    agg_score = scores['compound']
    final_sentiment = 1 if agg_score >= threshold else 0
    return final_sentiment

# apply lambda 식을 이용하여 레코드별로 vader_polarity( )를 수행하고 결과를 'vader_preds'에 저장
review_df['vader_preds'] = review_df['review'].apply( lambda x : vader_polarity(x, 0.1) )
y_target = review_df['sentiment'].values
vader_preds = review_df['vader_preds'].values

print(confusion_matrix( y_target, vader_preds))
print("정확도:", np.round(accuracy_score(y_target , vader_preds),4))
print("정밀도:", np.round(precision_score(y_target , vader_preds),4))
print("재현율:", np.round(recall_score(y_target, vader_preds),4))

[[ 6747  5753]
 [ 1858 10642]]
정확도: 0.6956
정밀도: 0.6491
재현율: 0.8514
```